

greedy

dijkstra

greedy

lec 7

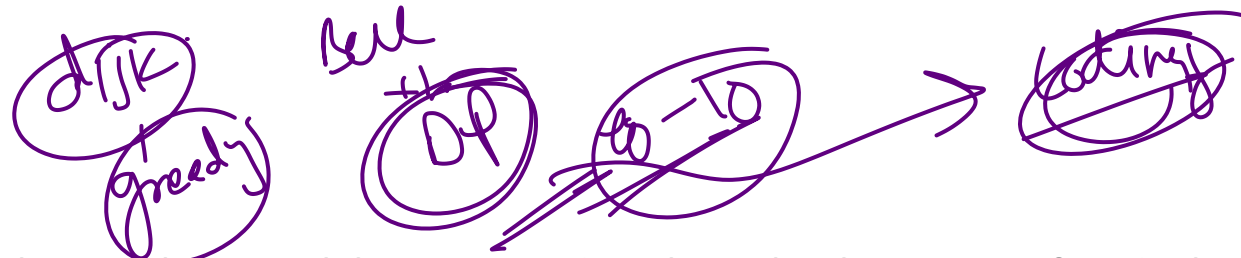
Dynamic programming

Lec_70

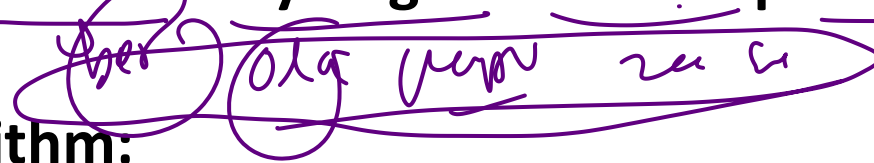
Bellman Ford Algorithm

greedy

Use:

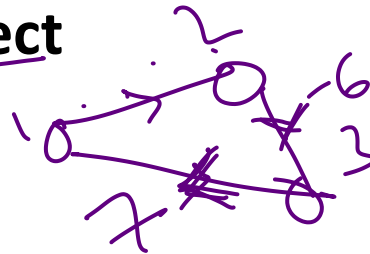


- The Bellman-Ford algorithm is like a guide that helps you find the shortest path from one city to all other cities, even if some roads have negative lengths. It's like a GPS for computers, useful for figuring out the quickest way to get from one point to another in a network



- In Dijkstra's Algorithm:

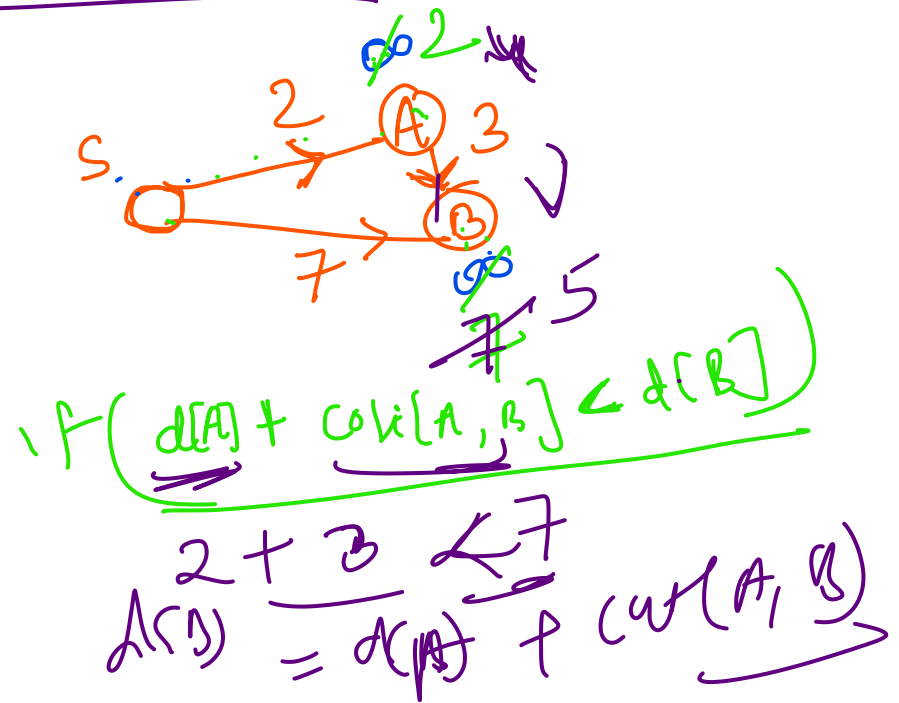
- If -ve edges but no -ve cycle than some times correct or incorrect
- if -ve cycle than always incorrect
- +ve edges (always correct)



Edge Relaxation :

- The Edge Relaxation property is defined as the operation of relaxing an edge $u \rightarrow v$ by checking whether the best-known way from S (source) to v is to go from S $\rightarrow v$ or by going through the edge $u \rightarrow v$. If it is the latter case we update the path to this minimum cost.
- Initially, the reaching cost from S to v is set infinite(∞) and the cost of reaching from S to S is zero.
- And the basic algorithm for Relaxation will be :

- if ($d[u] + c(u, v) < d[v]$) then
 $d[v] = d[u] + c(u, v)$
- where $d[u]$ represents the reaching cost from S to u
 $d[v]$ represents the reaching cost from S to v
 $c(u, v)$ represents reaching cost from u to v



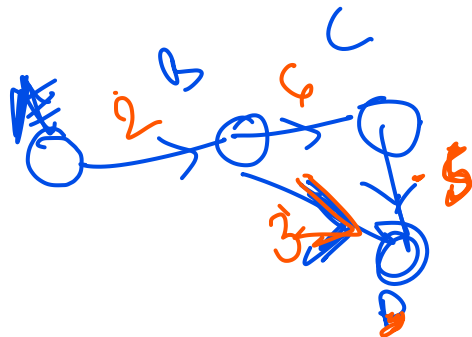
$O(V^2)$
 $O(V^3)$

Time complexity = $O(V^2)$ $\Rightarrow V = \text{vertices}$, $E = \text{edges}$
 $= O(V(V-1))$ $\Rightarrow \text{max } E = \frac{V(V-1)}{2}$
 $= O(V^2)$

$O(V^2)$
 $O(V^3)$

Working of bellman ford:

• Ex1:

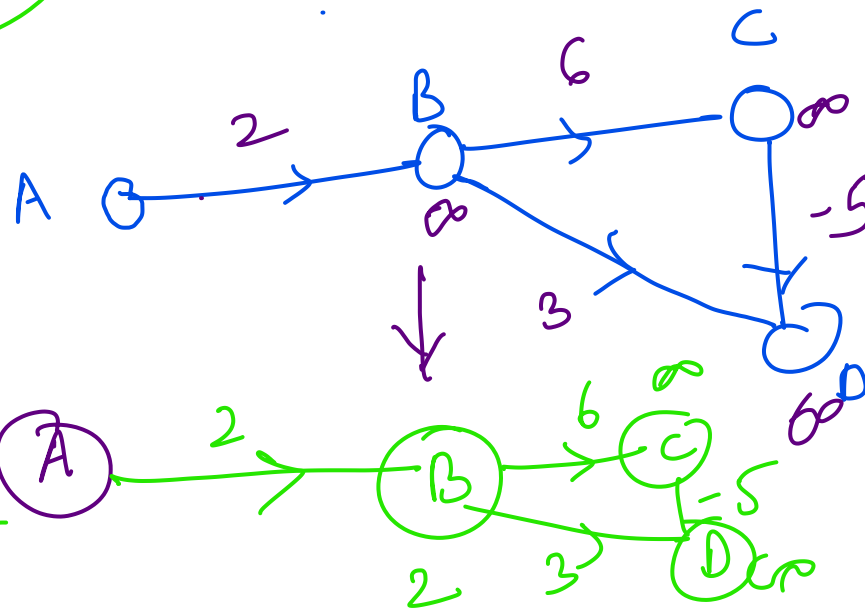


Source = A

If V vertices are
there in graph
need to do $n-1$ steps

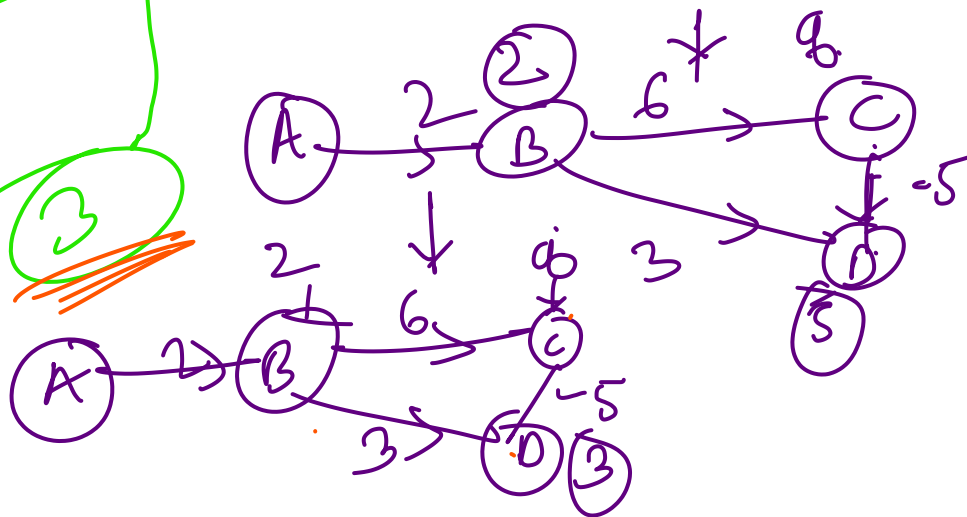


3



1

2

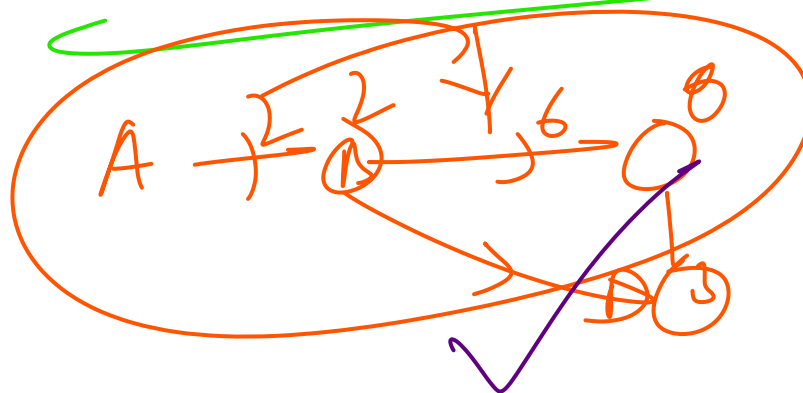


$A \xrightarrow{2} B$

$(A \rightarrow C) \Rightarrow \emptyset$

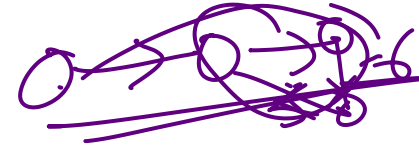
$A \rightarrow D \text{ (2)}$

4



Imp points:

is now to detect



- Bellman –Ford can detect –ve edge weight cycle if it is reachable from source

- To find SSSP in +ve edge weight graph we will use Dijkstra's algo because it will give always correct answer and Dijkstra's algo is faster algo than Bellman Ford

the

- If graph contain atleast one –ve edge weight than Dijkstra's may fail than we should go to bellman ford which is correct always

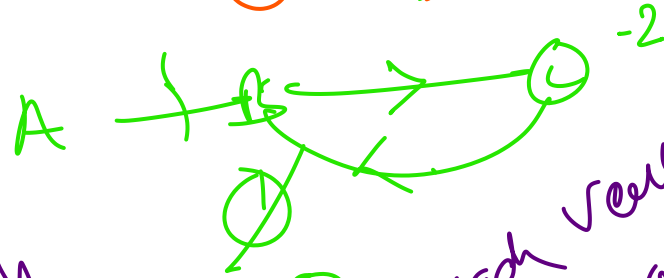
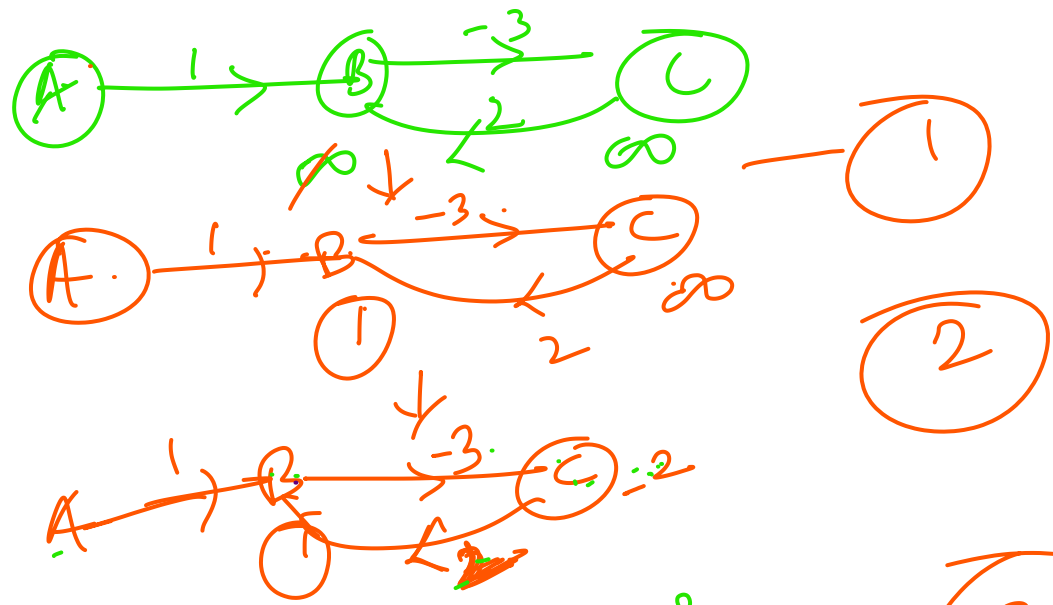
- ✓ If graph contain –ve edge weight cycle then bellman ford algo will find it if it is reachable

and will not print (inf)

$V \neq \infty$



go one more time so check
-ve cycle
and if
any change in
previous



value to each vertex
main -ve cycle is there



~~Sp~~ Wukh 2/8 Jan

Dynama

~~70X25~~
~~1750~~
~~60~~
~~50~~

